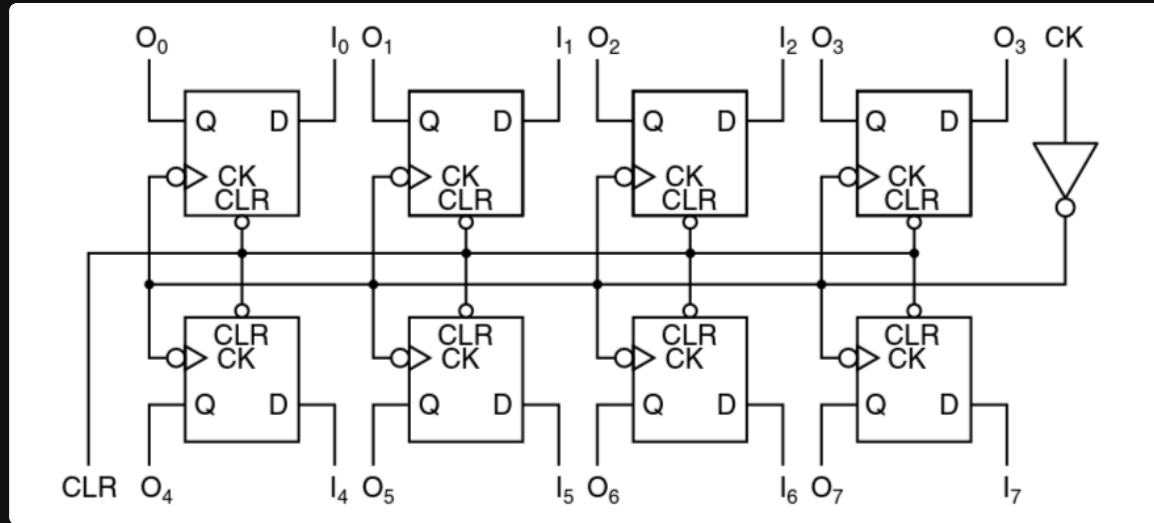


# Registers and Memory Organization

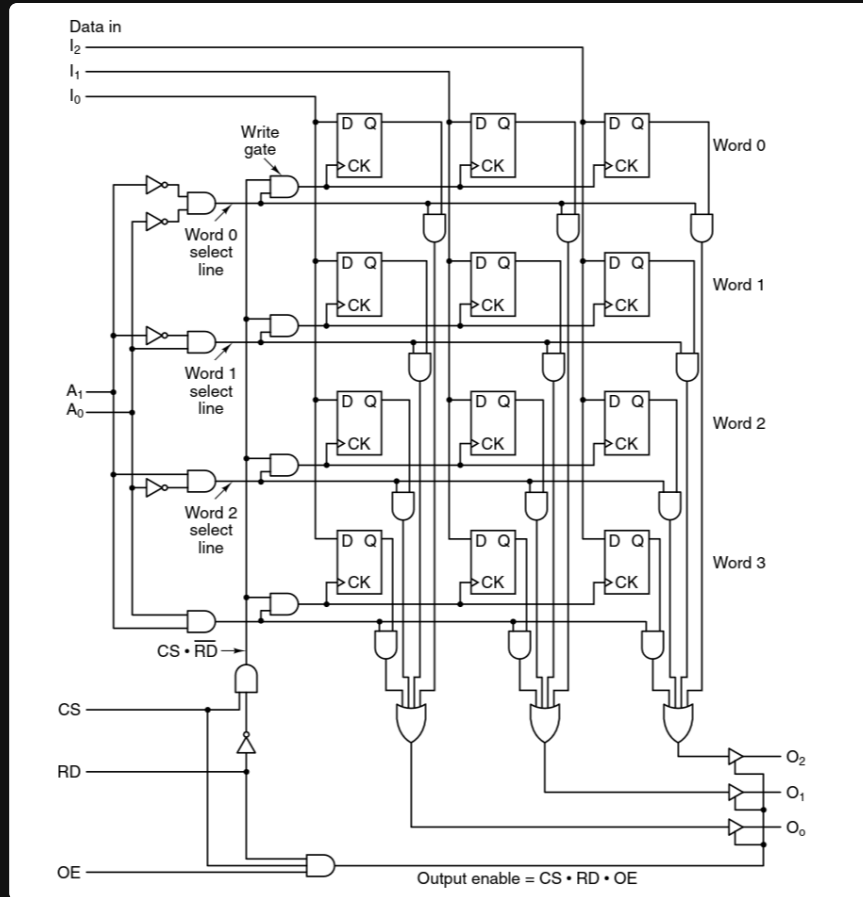
# Registers

Although we have now progressed from the simple 1-bit memory to the 8-bit memory,

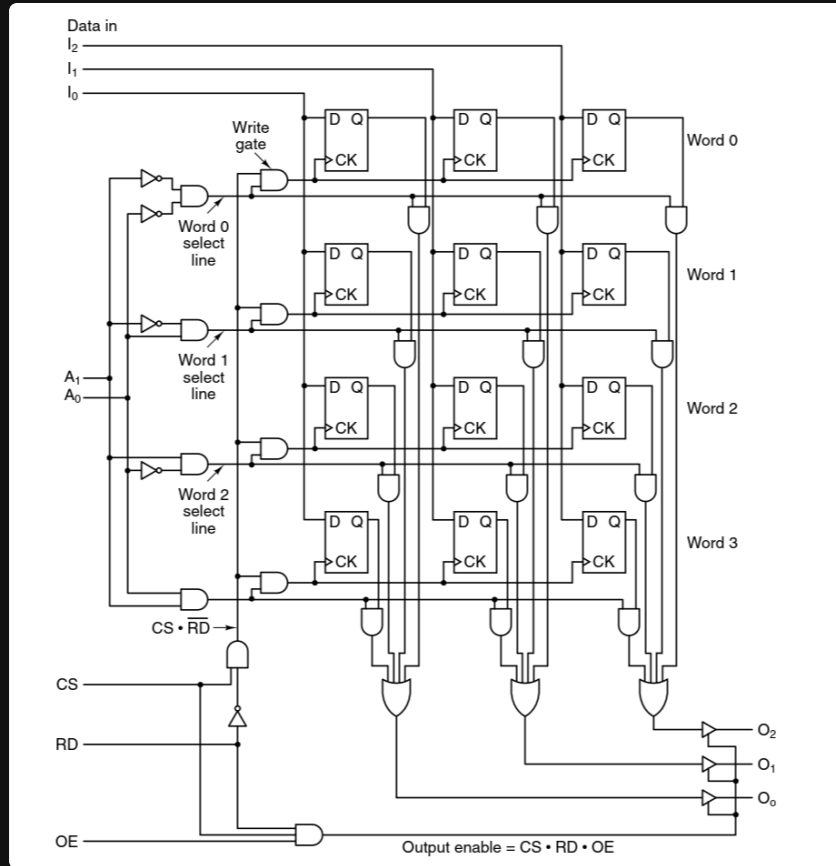


to build large memories a fairly different organization is required, one in which individual words can be addressed.

# Organization



# Organization



This is a memory with **four** 3-bit words.

Each operation reads or writes a full 3-bit word.

And while it's only *slightly* larger than our octal memory, it requires fewer pins.

Most importantly, the design **extends** easily to large memories.

# Inputs

While the memory may look complicated at first, it is really quite simple due to its regular structure.

It has **eight** input lines and **three** output lines.

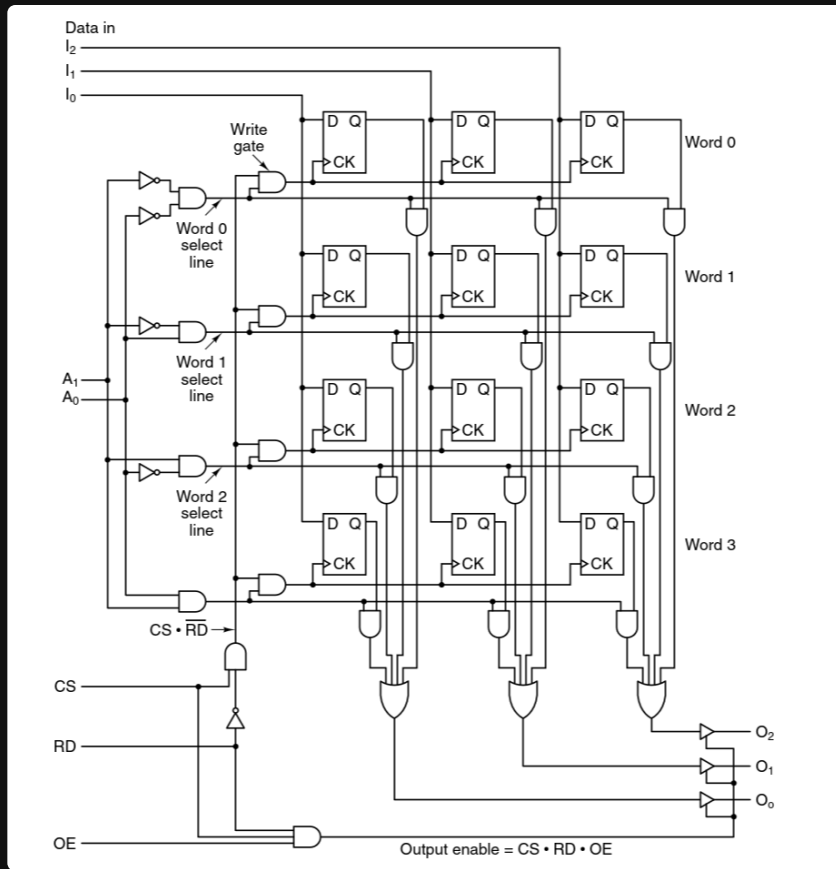
Three inputs are data:

- $I_0$ ,  $I_1$ , and  $I_2$ ,

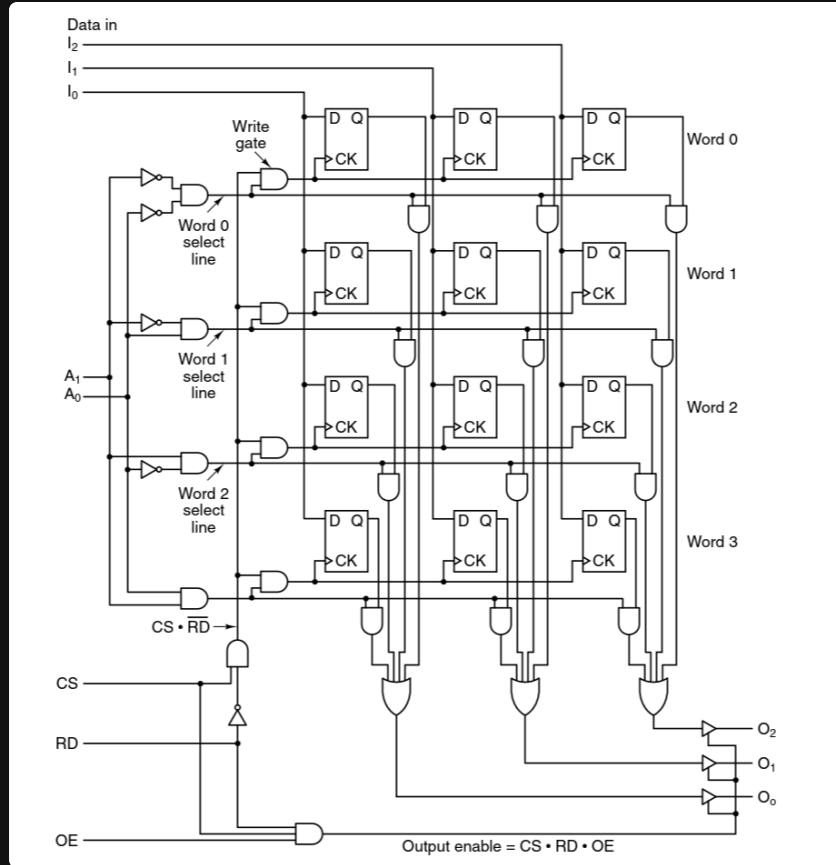
two are for the address:  $A_0$  and  $A_1$ , and

three are for control:

- $CS$  for Chip Select,
- $RD$  for distinguishing between read and write, and
- $OE$  for Output Enable.



# Outputs



The three outputs are for data: 00 , 01 , and 02 .

Our previous 8 -bit register requires 20 signals, including power and ground, while the 12 -bit memory requires only 13 signals.

In this memory, 4 memory bits each *share* one output signal.

The value of the **address lines** determine which of the 4 memory bits is **allowed** to input or output a value.

To select this memory block, external logic must set CS high ( 1 ) and also set RD high ( 1 ) for read and low (logical 0 ) for write.

# Reading and Writing

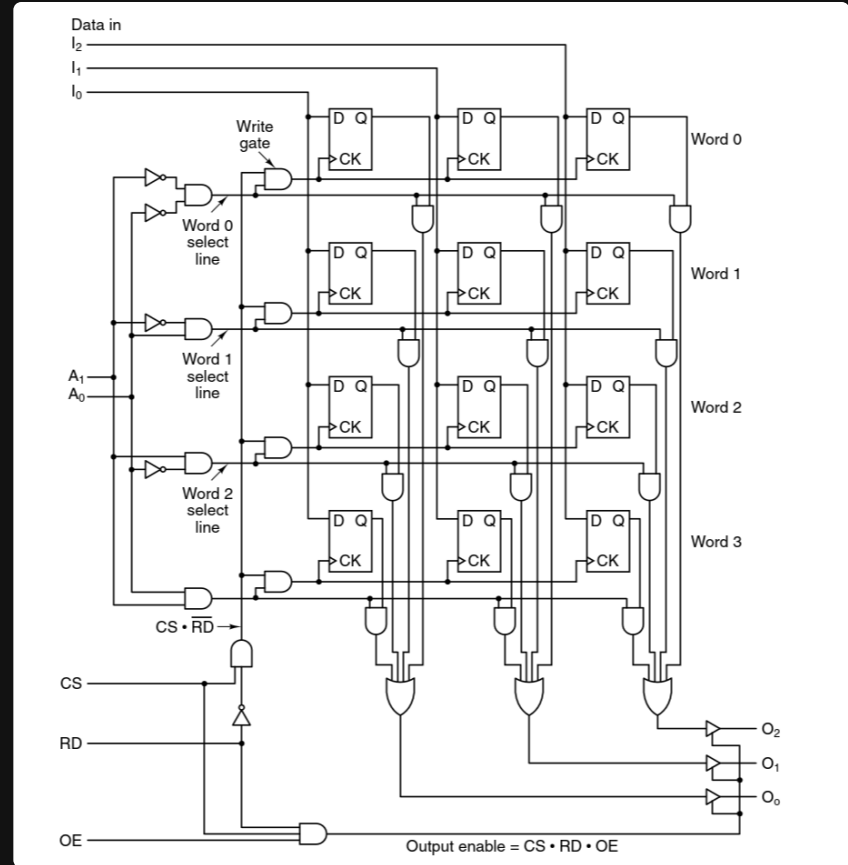
The two address lines **must be set** to indicate which of the four 3-bit words is to be read or written.

For a read operation,

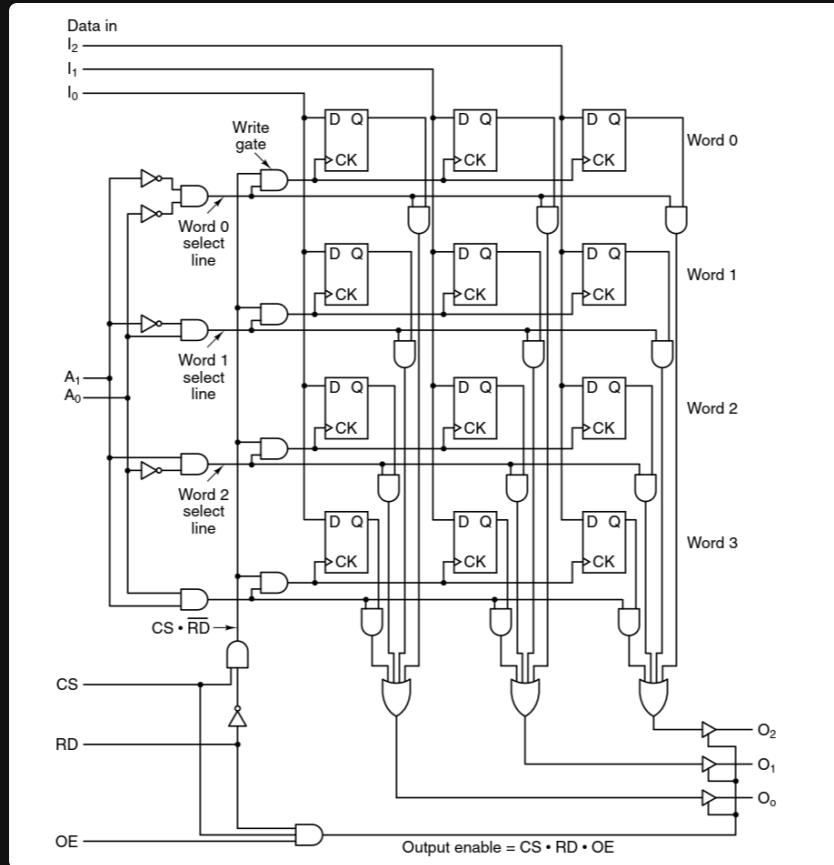
- the data input lines are not used,
- but the word selected is placed on the data output lines.

For a write operation,

- the bits present on the data input lines are loaded into the selected memory word
- the data output lines are not used.



# Writing



The four word-select AND gates at the left of the memory form a **decoder**.

They drive both the clock **input**, and the flip-flop **output**

When the chip has been selected for a write, the vertical line labeled  $CS \cdot \neg(RD)$  will be high (1),

- **Enabling** one of the four write gates, depending on which word select line is high.

This loads the input data into the flip-flops for that word.

A write is done only if **CS is high** and **RD is low**, and even then only the word selected by  $A_0$  and  $A_1$  is written

the other words are not changed at all.

# Reading

Read is similar to write.

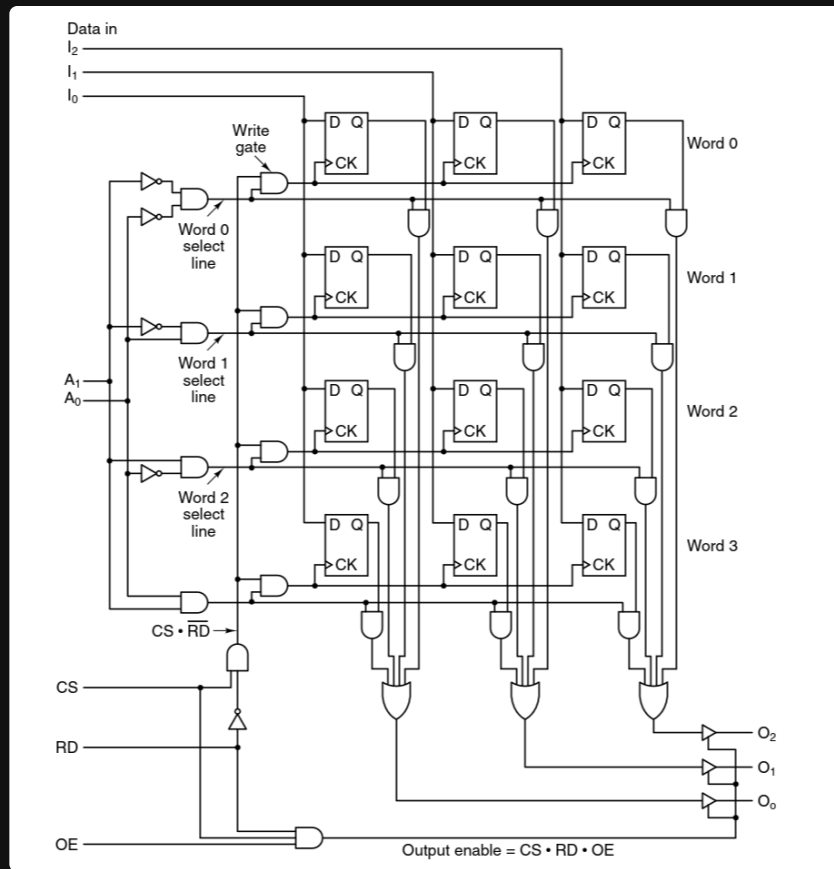
The address decoding is exactly the same as for write.

But now the  $\overline{CS} * \overline{!RD}$  line is low, so all the write gates are **disabled** and none of the flip-flops is modified.

Instead, the word select line that is chosen *enables the AND gates* tied to the `Q` bits of the selected word.

The output of the OR gates is identical to the value stored in the word selected.

The three words not selected make **no contribution** to the output.



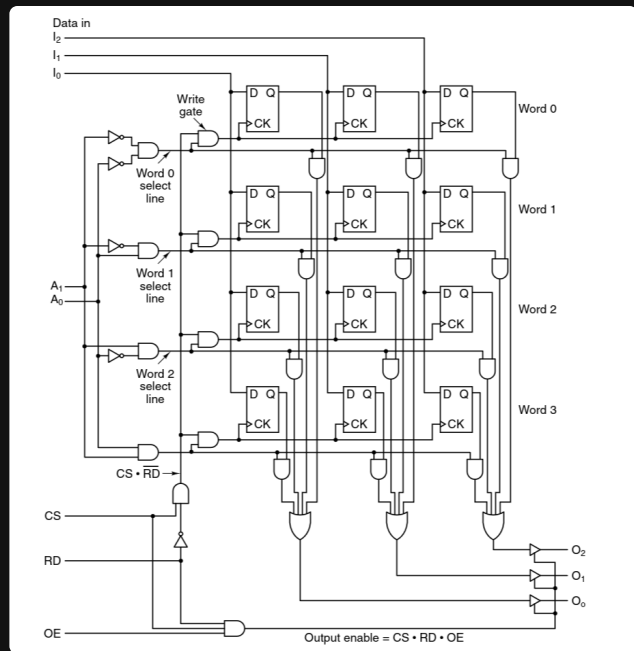
# Memory Chips

# Memory chips

We've made simple memory chips that can hold 4 words with 3 bits each.

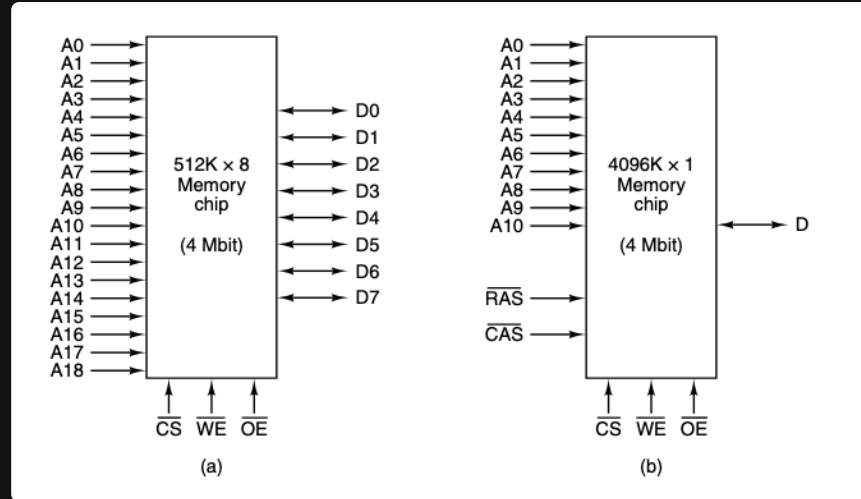
This structure, being as regular as it is, can also be **scaled up** to hold more words and more bits fairly easily.

And because integrated-circuit technology is well suited to making chips whose internal structure is repetitive, memory chips are an ideal application for it.



# Examples

Here are two ways to get 4 Mbit memory chips



These are organized slightly differently, and they each have their own advantages and disadvantages which the designer must take into account when creating memory.

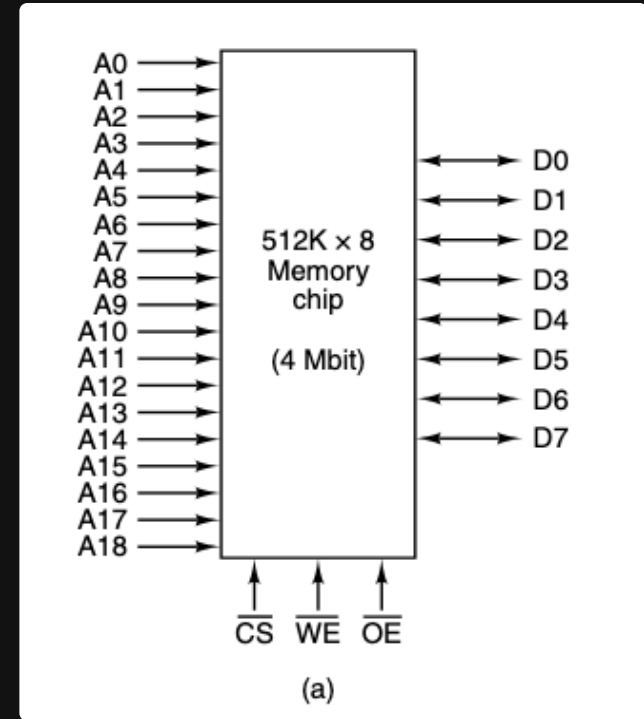
# 512Kx8

In this example,

We have a memory chip that can hold 512,000 words, each of which is 8 bits wide.

It requires

- 19 address lines to select one of the 512,000 words, and
- 8 data lines to read or write the 8 bits of data in that word.



# Terminology

Sometimes the high (1) voltage causes an action to happen, sometimes the low (0) voltage causes an action to happen.

To avoid confusion, the term `asserted` is used to mean that a signal is at the voltage level that causes the action to happen.

Thus, a *signal* named  $CS$  is asserted high, but a signal named  $\overline{CS}$  is asserted low.

Both mean an action has happened, but the voltage levels are different.

# 4096Kx1

In this memory chip, we have 4096,000 words, each of which is 1 bit wide.

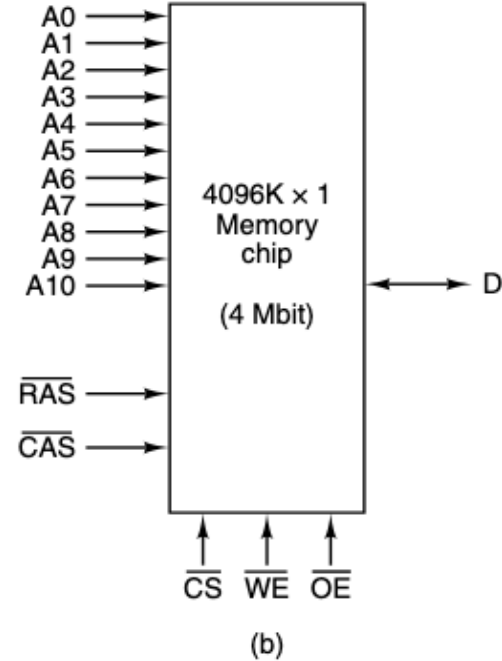
Internally this is organized as a 2048x2048 matrix of 1 bit cells.

And to address the chip, the first row is selected by putting its 11-bit number on the address pin,

then the  $\overline{RAS}$  (Row Address Strobe) is asserted.

After that, the column number is put on the address pins and the  $\overline{CAS}$  (Column Address Strobe) is asserted.

Finally, the data bit can be read or written on the single data pin.



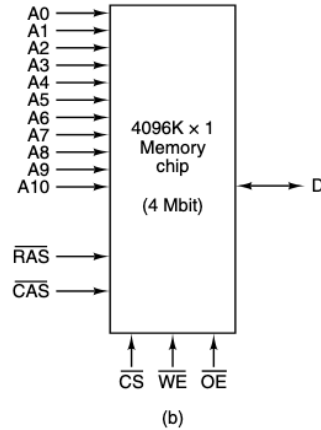
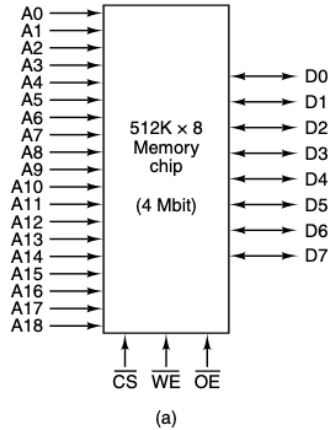
# 4096Kx1

Large memory chips are often constructed as  $n \cdot n$  *matrices* that are addressed by row and column.

This reduces the number of address pins needed, but increases the time it takes to access a memory cell since it now requires *two* steps instead of one.

To increase performance slightly, some memory chips can be given a row address followed by a sequence of column addresses to access consecutive bits.

# Signals

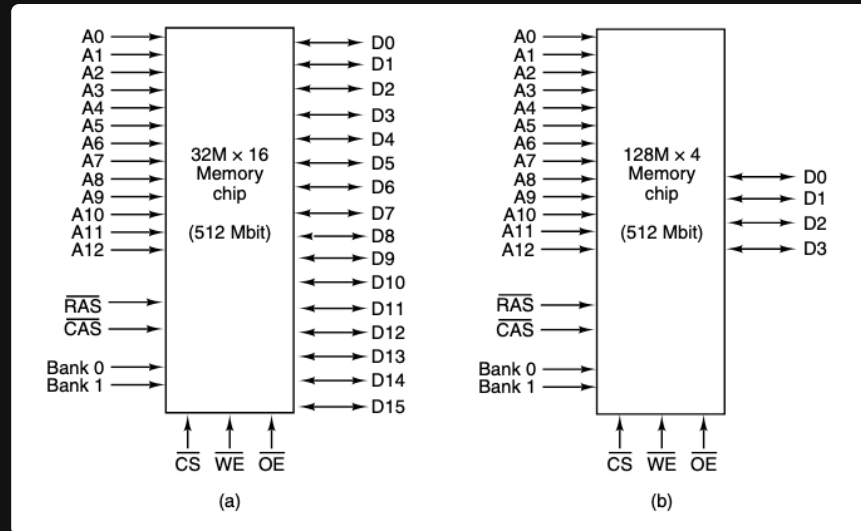


Both chips include some signals that help them work, such as

1.  $\overline{CS}$ , the chip select signal to select one of the many chips a computer might have
2.  $\overline{WE}$ , the write enable signal to indicate if data is being written or being read
3.  $\overline{OE}$ , the output enable signal, to drive the output signals

# More Examples

In this example, we have 2 more memory chips that both have 512Mbits of memory.



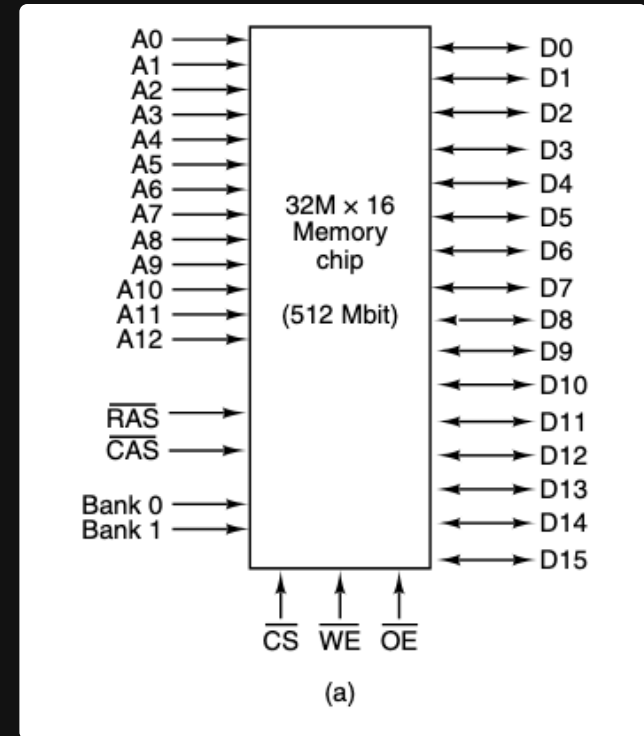
Internally, these chips both have 4 internal memory banks of 128Mbits each, which require 2 bank select lines to choose a bank.

# 32Mx16

In this example, the memory chip is a 32M x 16 design. With

- 13 lines for the  $\overline{RAS}$ ,
- 10 lines for the  $\overline{CAS}$ , and
- 2 lines for bank select

These 25 signals allow each of the internal 16 bit cells to be addressed

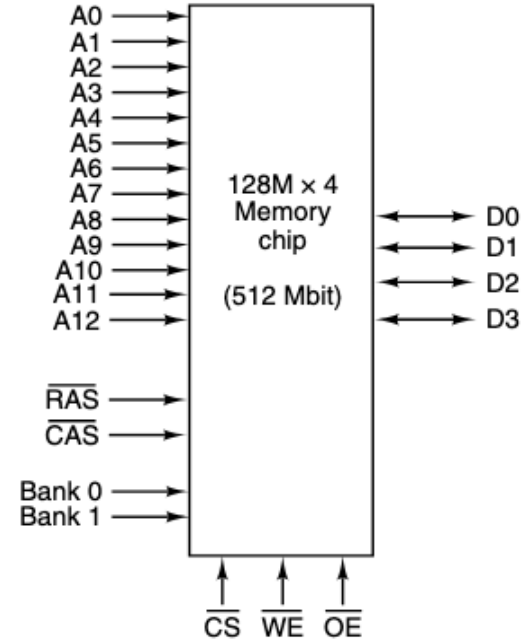


# 128Mx4

In contrast, the 128M x 4 design has

- 13 lines for the  $\overline{RAS}$ ,
- 12 lines for the  $\overline{CAS}$ , and
- 2 lines for bank select

Here, 27 signals are required



(b)

# Where do the RAS/CAS line counts come from?

Each bank's memory array is laid out internally as a grid of **rows × columns**.

DRAM chips use **address multiplexing** to save pins:

The number of RAS + CAS lines must add up to  $\log_2(\text{words per bank})$ :

- **32M × 16**: 512Mbit chip / 4 banks = 128Mbit per bank.

$128\text{Mbit} \div 16 \text{ bits} = 8\text{M words} = 2^{23} \text{ words per bank,}$

so  $\text{RAS} + \text{CAS} = 13 + 10 = 23 \text{ bits}$  ( $2^{13} \text{ rows} \times 2^{10} \text{ columns} = 8,388,608$ ).

- **128M × 4**: 512Mbit chip / 4 banks = 128Mbit per bank.

$128\text{Mbit} \div 4 \text{ bits} = 32\text{M words} = 2^{25} \text{ words per bank,}$

so  $\text{RAS} + \text{CAS} = 13 + 12 = 25 \text{ bits.}$

The exact row/column split (e.g. 13/10 vs 13/12) isn't fixed by the total alone, it reflects the physical row/column shape the manufacturer chose for the array

# Memory Chips

These examples illustrate some of the tradeoffs that must be made when designing memory chips.

How many columns and rows? Should the Matrix be square? Does the chip output 1, 4, 8, etc. bits at a time?

Should all the addresses have separate pins? Do you use rows and column strobes?

While we will not be making memory chips ourselves, the decision-making involved in their design will affect how we use them in lower level systems.

Rams And Roms

# Rams and Roms

- The memories we've discussed so far can all be **written to**. These types of memories are called **RAM s** (**Random Access Memory**).
- Technically speaking, *all* memory chips are randomly accessible, but the term RAM is too well established in the industry to change now.
- There are many types of RAMs, each with different characteristics:
  - Some are **obsolete** (e.g. FPM DRAM)
  - Others are still *actively used* despite their age (e.g. SRAM in CPU caches)

# Ram

There are 2 main varieties of RAM:

1. *Static* RAM (SRAM)
2. *Dynamic* RAM (DRAM)

Their main difference: whether they retain data indefinitely while power is on, or need to be refreshed periodically to keep their contents.

# Static Ram

Constructed using circuits similar to the basic **D flip-flop**, each bit is stored in a small latch made of a handful of transistors.

- Retains its data *as long as power is applied*
- **Extremely fast**: typical access times on the order of a *nanosecond* or less.
- Because of this speed, SRAM is the go-to choice for **CPU cache memory** (L1/L2/L3 caches), where every nanosecond of latency matters.



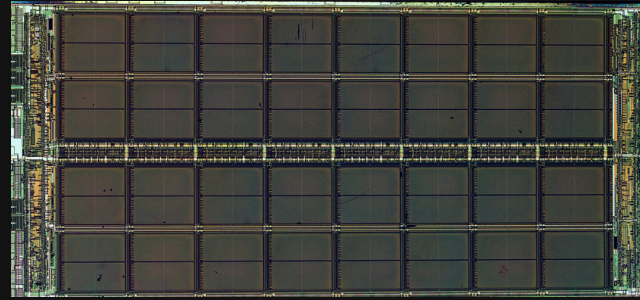
- Tradeoff: each cell needs more transistors, so SRAM is **larger and more expensive per bit** than DRAM.

# Dynamic Ram

Uses just a *single transistor and capacitor*\* (a "1T1C" cell) to store each bit, much smaller and cheaper than an SRAM cell.

- The capacitor's charge *leaks away over time*, so the data must be **refreshed periodically** (read out and rewritten) or it's lost.
- This refresh requirement means DRAM needs a **more complex controller/interface** compared to SRAM.

**Slower** than SRAM: typical access times are on the order of *10s of nanoseconds*.



But it has significantly higher **memory density** at a similar price compared to SRAM, you get far more bits per chip.

# A few other types of dynamic rams

- **FPM** (Fast Page Mode) DRAM - an older type of DRAM that allowed faster access to data within a single row.
- **EDO** (Extended Data Out) DRAM - allowed a second memory reference to *begin* before the previous one had finished completing.
- **SDRAM** (Synchronous DRAM) - synchronizes itself with the computer's system bus/clock, allowing faster, more predictable access to data.
- **DDR** (Double Data Rate) SDRAM - transfers data on **both** the rising and falling edges of the clock signal, effectively **doubling** the data rate of plain SDRAM.

This was followed by **DDR2**, **DDR3**, **DDR4**, and now **DDR5**, each generation improving on the last in terms of speed, bandwidth, and power efficiency.

**Example:** DDR4-3200 transfers roughly 3200 million data transfers/sec on a 64-bit bus, giving ~25.6 GB/s of bandwidth per module. DDR5 modules can roughly double that again.

# Roms

In many applications — toys,

- appliances,
- cars

The program and some data must remain stored even when the power is **turned off**.

Because RAM is built from transistors and capacitors, it **cannot retain data without power**.

**ROMs** (Read-Only Memory) solve this:

their contents are baked in at manufacturing time and **cannot be changed or erased** afterward.

They're made using photosensitive materials, which is exposed through a mask containing the desired bit pattern, then the exposed surface is etched away.

The only way to change a ROM's contents is to **manufacture a new chip** with a new mask.

Its significantly *cheaper* than RAM *per unit*, but only when produced in **large quantities**,

Since the one-time mask cost needs to be amortized across many chips.

# Proms

ROMs, while useful, are **inflexible**:

they can't be updated once manufactured, so development turnaround can span **weeks or months**, and the cost of a bugfix is high since a whole new mask must be created.

**PROM** (Programmable Read-Only Memory) solves this by letting the chip be programmed **once**, after manufacturing.

many PROMs contain an internal array of tiny **fuses**.

To "program" a bit, a specific fuse is selected by its row and column, then a **high-voltage pulse** is applied to a special pin, permanently blowing that fuse open.



**Example:** think of it like a grid of tiny fuses that all start intact (all bits = 1), programming the chip means selectively blowing fuses to set the bits you want to 0, once blown, a fuse can never be reconnected.

# Eproms

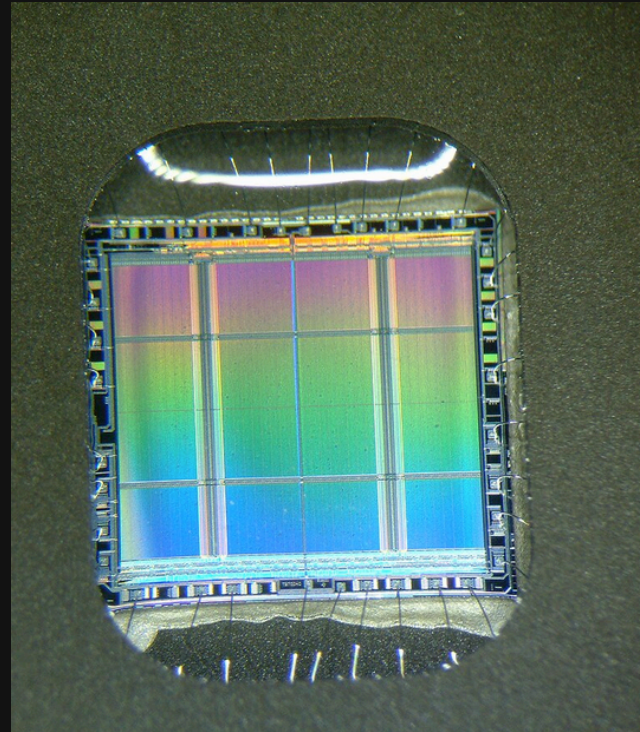
**EPROMs** (Erasable Programmable Read-Only Memory) extend the PROM idea by allowing the chip to be **erased and reprogrammed multiple times**.

They can be **field programmed** (in the field, not just at the factory)

and **field erased** by exposing the chip to strong **ultraviolet light** for about 15 minutes, which resets all bits back to 1.

**Example:** classic EPROM chips have a small **quartz window** on top of the package specifically so UV light can reach the die to erase it.

You can often spot one by that window, sometimes covered with an opaque sticker to prevent accidental erasure from sunlight.



# Eeproms

**EEPROMs** (Electrically Erasable Programmable Read-Only Memory) can be erased and reprogrammed multiple times using **only electrical signals**

no UV light required.

This allows **end users** to reprogram them directly, if needed, without special equipment.

- **Turnaround time** is much shorter than even EPROMs, since erasing is just an electrical operation.
- **Tradeoffs**: more complex, larger, and pricier per bit than EPROMs.



**Modern example: flash memory** (used in USB drives and SSDs) is a direct descendant of EEPROM technology, it erases and writes in larger blocks at a time rather than one byte at a time, which is what makes it fast and cheap enough for everyday storage.